UNIFIED REVIEW / PROOF-LANGUAGE DESIGN / LEAN

Nine Blueprints for a Mathematician's Proof Language over Lean

A unified review, convergence analysis, and assessment of nine independent design reports

In one paragraph. Nine agents were asked, independently and in parallel, to diagnose what makes Lean proofs longer and less natural than a mathematician's argument, and to design a better language that stays rigorously checkable. This document reads all nine reports in full, tallies forty design commitments across them, and finds an unusually strong consensus: a Lean-hosted layer whose unit is a typed mathematical step with a contract, a frozen theorem statement, a scoped obligation ledger, certified representation views, a synthesis boundary that never turns search failure into refutation, and search-free replay. It then tests the shared diagnosis against the whole ProveIt corpus rather than the twenty-three files the reports sampled, identifies where the reports differ, assesses which ideas are strong, weak, or under-specified, and adds several points none of the reports make: that almost all proposed *methods* already exist as Mathlib tactics so the missing piece is an accounting layer, that Mathlib's `says` combinator is already a miniature of the discovery/replay split, that contracts should be annotated theorem types rather than a new method language, and that definitions, not proofs, are the larger lever for "natural expression."

Prepared by Claude (Anthropic) for Vladimir Reshetnikov

5 September 2026

Status. A synthesis of design documents, plus a text-level audit of an existing Lean corpus. No proof language was implemented, no Lean code was compiled, and no claim below about future usability has been measured. Tallies of what each report says are this reviewer's readings of the sources and may be disputed at the margin; the sources are cited so that every tally can be checked.

Abstract

Nine reports under `docs/ideas` propose languages named MathStep, Contour, Loom, MPL ("Mathematical Intent, Formal Evidence"), Mosaic, Motive, Outline ("Proofs at the Scale of Ideas"), Reason, and Spine. Each is grounded in a small sample of the ProveIt repository and in the architecture of Leant. This review establishes three things. First, the reports converge on a single architecture: of forty design commitments extracted from the texts, twenty-nine are held unanimously and five more by at least seven of nine; the spread that remains concerns specific method families and synthesis engineering, not the design. Second, the shared diagnosis survives a quantitative check. A heuristic audit of the 1,004-file FabiusFunction development (about 90,000 tactic invocations) finds that roughly a third of tactic invocations are representation, side-condition, or index bookkeeping, that this share is flat across proof lengths, and that long proofs are already organised as chains of `have` assertions, which is exactly the shape the proposed languages want to make primary. Third, the reports share blind spots. Nearly every method they propose exists in Mathlib at the pinned toolchain (`gcongr`, `positivity`, `push_cast`, `zify`, `fun_prop`, `filter_upwards`, `wlog`, the `Finset.sum_bij` family), so the deliverable is an accounting layer over existing automation, not new automation; Mathlib's `says` combinator already implements the discovery-versus-replay split at the tactic level; and none of the reports treats definitions, elaboration performance, or instance management as sources of friction, although the corpus shows all three. The review closes with an assessment of each major idea, a consolidated adversarial test suite, a phased recommendation, and per-report summary cards.

Contents

1 Introduction

1.1 The task and the material

The nine reports were produced independently, in parallel, from the same brief: analyse the shortcomings of Lean as a language for expressing mathematics, and brainstorm a better language that lets mathematicians express ideas concisely and naturally while remaining amenable to rigorous formal verification. Each report was asked to ground its analysis in Vladimir Reshetnikov's ProveIt repository (a large Lean 4 development, primarily around the Fabius function) and in Leant, an experimental Djex-based synthesis REPL for Lean 4. Table 1 lists them.

Name	Directory under `docs/ideas`	ProveIt files sampled	Pages	Extra artifacts
MathStep	`MathStep`	AutonomousIteratedDeriv, AlgebraicInverseGermAnalytic, AliasQBinomialBridge, NaturalDeduction/Calculus	42	source manifest
Contour	`contour-proof-language`	BinomialInversion, LambertWElementaryBounds, LagrangeInversionUniqueness, PascalTailIdentity	38	`grammar.ebnf`
Loom	`loom_proof_language`	AutonomousIteratedDeriv, AbelPolynomialSeries, AffineDifferenceOrbit	30	
MPL	`mathematical_proof_language`	AbelPolynomialSeries, AutonomousIteratedDeriv, AssociatedStirling, JacobianConjecture/Scaling	35	
Mosaic	`mosaic_proof_language`	SharpFlatness, Regularity, FloorSqrtSum	32	`build.sh`
Motive	`motive_proof_language_article`	BinomialInversion, LambertWElementaryBounds, GeometricRichardson	30	
Outline	`outline_proof_language`	Basic, MeanValueBracket, QPascalSummation, Arithmetic, FloorSqrtSum	34	
Reason	`reason_proof_language`	Basic, Arithmetic, AffineDifferenceOrbit, FloorSqrtSum	32	executable toy checker, 32 tests
Spine	`Spine_Proof_Language`	ThueMorsePrefix, AffineIndependentCopy, AlgebraicInverseGerm	39	build scripts

Table 1: The nine reports. Page counts are those of the shipped PDFs. All nine are dated 5 September 2026 and all pin ProveIt commit `24ce8bd7`; Loom, MPL and Spine pin Leant commit `3a40904b`, the others `c36adf11`, and Reason records no Leant commit.

Two facts about the material shape how the rest of this review should be read. The reports draw on twenty-three distinct ProveIt files, and their samples are almost disjoint: only six files are shared by more than one report. That makes agreement on *which kinds of friction recur* informative, because the agreement was reached on different evidence. On the other hand, eight of the nine PDFs carry the same author metadata ("OpenAI" or "ChatGPT"), all nine are dated the same day, and all answered the same prompt. Their convergence is therefore best read as robustness of one model family's design instincts under resampling of the evidence, not as nine independent expert opinions. Section 7.7 returns to what the convergence does and does not establish.

1.2 Method of this review

Every `.tex` source was read in full. From the texts I extracted forty design commitments (Section 4.1), scored each report on each, and collected each report's worked examples, negative tests, proposed method names, grammar, and roadmap. Where the reports quote or characterise Leant, I verified the claims against the local Leant checkout at commit `3a40904b`; the characterisation of the `Verified` wrapper, the fragment translator, and the candidate-quality and rewrite-analysis documents is accurate in every report that makes it.

Because a local ProveIt checkout was available, I added something none of the reports could do at the time: a text-level audit of tactic usage across the entire FabiusFunction development (Section 3). The checkout is at commit `7c4e3f10`, which is newer than the commit the reports pin; all twenty-three sampled files are present at the newer commit. The audit script and its output are shipped beside this document.

Nothing was compiled. No proposed syntax was executed. Where I give a verdict on an idea, the verdict is an argument, not a measurement, and it is labelled as such.

1.3 Summary of findings

1. **The diagnosis is shared and specific.** All nine locate the problem in granularity, not foundations: the unit of interaction in Lean is a tactic, while the unit of mathematical communication is a step such as "reindex the sum" or "differentiate the identity on the open set." All nine classify the resulting verbosity into essentially the same four bins, and all nine find at least one existing proof that is already ideal and should not be touched.
2. **The architecture is nearly unanimous.** Twenty-nine of forty commitments are held by all nine reports (Section 4.1). The unit of authorship has nine names and one definition. The nine "central contract" slogans are paraphrases of one sentence.
3. **The corpus supports the diagnosis, with a correction.** Across about 90,000 tactic invocations, roughly 32% are bookkeeping of the kinds the reports want to reconstruct; the share is flat across proof lengths; and 32% of invocations are author-stated assertions, rising to 39% in proofs over thirty lines. The correction is that analysis plumbing (filters, derivatives, integrals) is under-represented in the reports' samples relative to the corpus by a factor of about 2.5.
4. **Most proposed methods already exist.** At the pinned toolchain, Mathlib provides a tactic or lemma family for nearly every method the reports name. The deliverable is an accounting layer: contracts, ledger, seal, explanation. Mathlib's `says` combinator already realises the discovery/replay split for single tactics, and the corpus does not use it.
5. **The nine soundness arguments are one lemma**, and it is not where the risk is. The risk is statement fidelity, cost predictability, and explanation, on which no report has a theorem and all nine say so.
6. **Shared blind spots.** Definitions and API design, type-class instance management, elaboration performance, and the fact that all nine keep theorem *statements* in Lean, so that "natural expression" is promised for proofs only.

2 The shared diagnosis

2.1 Granularity, not foundations

The reports agree that Lean's kernel, its elaboration architecture, its tactic framework, and Mathlib are not the obstacle. Contour says the reviewed code "does not suggest that Lean's foundations are the main obstacle" but "a mismatch in granularity"; MathStep warns against making Lean a straw man; Motive stresses that Lean's implicit arguments already perform substantial reconstruction; Spine calls its own proposal "a conservative foundation, an ambitious

interface." Every report keeps Lean's term language for formulas, Lean's kernel as the checker, and Lean's declarations as the mathematical library.

What they blame is the *default unit of interaction*. A mathematician writes "differentiate the induction hypothesis and use the chain rule"; the Lean author writes a filter equality via `Filter.eventuallyEq_of_mem`, a derivative-congruence rewrite, a chain-rule composition, and a rewrite by the recurrence. The sentence and the script contain the same mathematics; the script additionally contains the library's representation choices. This is the diagnosis in all nine reports, illustrated in every one by a fragment of real ProveIt source.

2.2 Four kinds of detail

Each report proposes a taxonomy of what makes proofs long. Table 2 aligns them. Names differ, but the partition is stable: a mathematical decision that must stay visible; a hypothesis whose *existence* is essential even if its *proof* is routine; a representation bridge that is reconstructible; and interface trivia that should disappear.

Report	Keep visible	Essential but reconstructible	Representation	Interface trivia
MathStep	essential mathematical content	routine derivation	representational friction	library-interface friction
Contour	mathematical decision	scoped side conditions	representation management	logical assembly, method implementation
Loom	mathematical commitment	reconstructible detail	representation management	interaction overhead
MPL	strategic detail	semantic detail	representational detail	incidental detail
Mosaic	logical structure	mathematical infrastructure	representation work	search residue
Motive	mathematical decision	representation obligation	representation obligation	interaction overhead
Outline	structural reasoning, discovery	local calculation, index management	representation	(same)
Reason	logical structure	mathematical side conditions	representational overhead	(same)
Spine	mathematical cost	deductive cost	representation cost	(same)

Table 2: Nine taxonomies of proof length, aligned to a common four-way partition. Where a report uses three bins, the fourth column repeats the third.

Every report also makes the same qualification: the partition is contextual. A cast identity can be the point of a theorem about truncated subtraction; a branch condition that is automatic in one section is a new hypothesis in another. Hence all nine reject a fixed global classification and propose *expandable* presentation instead, with the reconstruction retained in a checkable form.

2.3 The already-short control

A striking shared move is that each report went looking for a counterexample to its own thesis and found one. Table 3 lists the declaration each report identifies as already ideal Lean. The conclusion drawn is identical in all nine: a new language must not make good Lean longer, and a fair evaluation must compare against *refactored* Lean with the same helper lemmas, not against the original script.

Report	Declaration found already ideal	Lesson drawn
MathStep	the algebraic-branch derivative as a syntax-directed method	compare against expert Lean using the same lemmas
Contour	`le_log_one_add_mul_exp` (a readable `calc`) and single-expression corollaries	"some existing Lean is already ideal"
Loom	the autonomous-equation module "is already economical"	a helper library may give most of the benefit
MPL	`counterexample_scaling`: `funext; fin_cases; simp; field_simp`	"existing Lean can already be very concise"
Mosaic	the absolute-difference Lipschitz corollary	the diagnosis is mixed, not sweeping
Motive	normalized Richardson polynomial: `rw` then `div_self`	the language must earn its place where bridges repeat
Outline	the inverse-error corollaries in MeanValueBracket	separate library, refactoring, automation, and language gains
Reason	the Arithmetic file already extracts small reusable lemmas	do not compare only with unoptimised Lean
Spine	`existsUnique_germRoot` and a forward-difference proof	a language should not force short proofs into prose

Table 3: Negative controls found by each report.

2.4 Friction families and the evidence behind them

The most informative convergence is not in the taxonomy but in the concrete recurring operations. Table 4 groups the worked examples by the mathematical operation whose formal implementation the report found disproportionate. Because the file samples are nearly disjoint, a family that appears in most reports appears in many different files.

Two smaller coincidences deserve mention because they are evidence of the same kind. All three reports that sampled `FloorSqrtSum.lean` (Mosaic, Outline, Reason) independently chose to re-prove the floor-square-root sum by double counting the set $\{(j,k) : j^2 \le k \le n\}$, precisely so that the divisibility-by-six and non-truncation conditions become consequences of a counting balance rather than facts to be guessed. And four reports (MathStep, Loom, MPL, Reason) use the identical two-function counterexample $f(t) = t$, $g(t) = 0$ at $t = 0$ to explain why equality at a point cannot license differentiation. When independent designers reach for the same alternative proof and the same counterexample, the underlying friction is real and the intended contract is well-defined.

3 Testing the diagnosis against the corpus

The reports sampled twenty-three files, about five thousand lines. The FabiusFunction development alone has 1,004 Lean files and roughly 249,000 non-blank, non-comment lines. This section reports a text-level audit of that whole subtree, using a small Python script shipped in `docs/unified_report/tools/`. The script strips comments, finds every `theorem` or `lemma`, and classifies each tactic invocation in its proof (a line whose first token is a tactic keyword, or a `calc` step) into one of ten families chosen to mirror the reports' taxonomy. The families and the classification rules are in Appendix C. The audit is a heuristic on source text: it cannot see what a tactic did, it classifies a `rw` by the lemma names it mentions, and it counts a multi-line `have` as one assertion plus whatever tactic lines its sub-proof contains. Its purpose is to estimate proportions and to make the reports' qualitative claims falsifiable, not to certify anything.

Friction family	Reports	Files in which it was observed
Cast and scalar transport ($\mathbb{N} \to \mathbb{Z} \to \mathbb{Q} \to R$, `algebraMap`, `push_cast`)	9 of 9	AliasQBinomialBridge, BinomialInversion, AbelPolynomialSeries, AssociatedStirling, FloorSqrtSum, Arithmetic, ThueMorsePrefix, LagrangeInversionUniqueness
Finite-sum index management (range vs. interval, endpoint splitting, reindexing, congruence under a binder)	9 of 9	BinomialInversion, PascalTailIdentity, AffineDifferenceOrbit, QPascalSummation, ThueMorsePrefix, AssociatedStirling, AliasQBinomialBridge, FloorSqrtSum
Ordered-field side conditions (positivity, nonzero denominators, sign when multiplying or cancelling, unit vs. nonzero)	9 of 9	LambertWElementaryBounds, GeometricRichardson, AlgebraicInverseGermAnalytic, SharpFlatness, MeanValueBracket, Scaling, AbelPolynomialSeries, AlgebraicInverseGerm
Induction motive and generalisation	9 of 9	AutonomousIteratedDeriv, AffineDifferenceOrbit, SharpFlatness, QPascalSummation, PascalTailIdentity, ThueMorsePrefix, Arithmetic
Local derivative and filter plumbing (open set to neighbourhood equality, chain rule, limits by continuity)	7 of 9	AutonomousIteratedDeriv, AffineDifferenceOrbit, AffineIndependentCopy, MeanValueBracket, Regularity
Symmetry and "without loss of generality" as a checked reduction	8 of 9	Regularity, MeanValueBracket, Basic (evenness); theoretical in the rest
Formal power series and exponential generating functions (coefficient extraction, `HasSubst` admissibility)	4 of 9	LagrangeInversionUniqueness, AbelPolynomialSeries, AssociatedStirling, AlgebraicInverseGerm

Table 4: Recurring friction families. Files are named without their directory prefix. The count is of reports that treat the family in a worked example or a dedicated section.

3.1 What the corpus looks like

Call the union of OBLIG, CAST, INDEX, REPR, NORM, SIDE and ANALYSIS the *reconstructible* families: they are the invocations the nine reports want to move from the primary text into an expandable ledger. In the corpus this union is 32.3% of all tactic invocations; in the reports' sample it is 34.1%. The sample is representative in aggregate, and the reports' central quantitative intuition, that around a third of proof text is bookkeeping, is supported.

3.2 What the audit says about the reports

Observation 3.1 (The reconstructible share is flat)**.** The reconstructible share is 31–36% in every length bucket. Bookkeeping is not concentrated in long proofs; it is a constant tax. This favours the reports' proposal of a *uniform* step layer over a special tool for hard proofs.

Observation 3.2 (The corpus is already spine-shaped)**.** Author-stated assertions are 32% of all invocations and 39% in proofs over thirty lines; `have` is the single most common tactic head (22,919 uses), and the most common bigram is `have` followed by `have` (8,111 times). Three quarters of all tactic work sits in proofs longer than ten lines, and those proofs are organised as chains of assertions with local sub-proofs. The nine reports describe a language whose primary text is a chain of typed assertions; the corpus shows that experienced Lean authors already write that way inside `by` blocks. The proposed layer would formalise an existing practice, not impose a new one.

Family	What it counts	Corpus (%)	Corpus (count)	Sample (%)	Sample (count)
ASSERT	`have`, `obtain`, `calc` steps, `let`, `set`, `suffices`	31.7	28,727	27.0	359
REWRITE	`rw`/`simp` with ordinary library lemmas	19.5	17,715	20.8	277
STRUCT	`intro`, `refine`, `apply`, `exact`, `cases`, `induction`, …	16.4	14,892	18.1	240
ANALYSIS	`filter_upwards`, `fun_prop`, filter/derivative/integral lemmas	7.2	6,524	2.9	38
REPR	`change`, `show`, `unfold`, `dsimp`, `funext`, `ext`, `congr`, `convert`	5.2	4,688	5.6	74
INDEX	finite-sum, range/interval, and cardinality lemmas	4.8	4,338	7.1	94
NORM	`ring`, `field_simp`, `linear_combination`, `abel`	4.4	3,965	3.5	47
SIDE	`omega`, `linarith`, `positivity`, `norm_num`, `gcongr`, `bound`, `decide`	4.3	3,941	6.2	82
CAST	`push_cast`, `exact_mod_cast`, `zify`, cast and `map_*` lemmas	4.0	3,604	5.9	78
OBLIG	one-line `have ... := by <decision procedure>`	2.5	2,243	3.0	40
Total	tactic invocations	100	90,637	100	1,329

Table 5: Tactic invocations by family, whole FabiusFunction subtree (1,004 files, 12,322 detected theorems, 1,300 of them term-mode) versus the twenty-three files sampled by the reports (274 theorems, 45 term-mode).

Proof length (lines)	Theorems	Invocations	Share of all invocations	Recon. share	ASSERT	REWRITE	STRUCT
≤ 3	3,633 (29.5%)	3,483	3.8%	32.5%	5.1%	40.9%	21.5%
4–10	4,739 (38.5%)	19,140	21.1%	35.8%	20.9%	23.3%	20.1%
11–30	2,952 (24.0%)	34,457	38.0%	31.9%	33.1%	18.4%	16.6%
> 30	998 (8.1%)	33,557	37.0%	30.8%	39.2%	16.4%	13.6%

Table 6: The same corpus by proof length. Median proof length is 6 lines, mean 11.6, 90th percentile 27, maximum 428.

Observation 3.3 (The reports' samples under-weight analysis)**.** ANALYSIS is 7.2% of the corpus but 2.9% of the sample, while INDEX, CAST and SIDE are each over-represented by roughly 40%. The corpus contains 868 `filter_upwards` invocations, 645 references to `intervalIntegral`, and 303 to `Filter.Eventually`; the sampled files are shorter (median 4 lines against 6) and more combinatorial. The consequence for method priorities is direct: the locality/derivative-transfer and limit-by-continuity contracts, which only four reports develop in detail, deserve at least the priority given to finite-sum reindexing. MOTIVE's warning that analytic moves are where omitted hypotheses carry the real difficulty is, on this evidence, the more important half of the design.

Observation 3.4 (The routine-discharge portfolio is empirical)**.** Of the 2,243 one-line obligations of the form `have h : P := by <tactic>`, the tactic is `omega` in 646 cases, `positivity` in 497, `linarith` in 305, `exact_mod_cast` in 257, `ring` in 159, `simp`/`simpa` in 170, `norm_num` in 67, `nlinarith` in 55. Every report proposes a "routine" or "local" profile without saying what is in it; the corpus says what is in it.

Observation 3.5 (Bridge density varies six-fold between files)**.** Among files with at least forty invocations, the reconstructible share ranges from 10% (`StirlingParityBitwise`, `ThueMorseLucasSupport`) to 71% (`ThueMorseBlockProducts`, `PerronRootEnclosure`); the eight highest are exponential-generating-function, umbral, and Stirling-number files (`BellShiftEGF`, `TouchardShiftEGF`, `BellUmbra`, `StirlingFirstReverse`). This is independent support for the EGF coefficient interface proposed by LOOM and MPL: it is where bookkeeping is densest. The most-used cast lemmas are `smul_eq_mul` (249), `map_mul` (205), `map_sub` (140), `map_one` (137), `map_add` (108), `Nat.cast_one` (91), `zpow_natCast` (88), `map_sum` (84), `map_natCast` (77), and `Nat.cast_sub` (69), which is the vocabulary a transport view must own.

Marker (FabiusFunction subtree)	Count	Relevance
`sorry` in code	0	the corpus is complete; the four textual hits are docstrings saying "sorry-free"
`native_decide`	11 (2 files)	the version-aware axiom allowlist the reports demand is directly applicable at Lean 4.32
`set_option maxHeartbeats`	15 lines, 14 files	elaboration cost is a real, if modest, source of friction that no report discusses
`gcongr`	141	Contour's "multiply the inequality by a positive factor" move already exists
`linear_combination`	269	certificate-style algebra is already in use
`letI`/`haveI`	162	Spine's "local instances are evidence" point is measurable
`says`, `exact?`, `simp?`, `#print axioms`, `aesop`, `polyrith`	0	the discovery/replay and axiom-audit mechanisms Lean already has are unused here
`grind`	6	

Table 7: Markers relevant to trust, performance, and existing mechanisms. Repository-wide, `maxHeartbeats` appears on 9,865 lines in 7,406 files, almost all in the Computability directory, which appears to contain generated developments.

3.3 Caveats

The audit is text-level. It cannot distinguish a `rw` that applies a deep theorem from one that flips an equation; it puts both in REWRITE, which is why that family is best read as "library-interface work of unknown depth." It classifies a `simp` without recognisable lemma names as REWRITE. The theorem detector may miss declarations with unusual attributes and counts a few definitions' proofs of well-foundedness. The corpus commit is newer than the one the reports pin, so file-level numbers for the twenty-three sampled files may differ slightly from what the reports saw. None of this changes the proportions materially; all of it is reproducible from the shipped script.

4 The consensus architecture

4.1 Forty commitments, nine reports

Table 8 scores each report on forty design commitments extracted from the texts. A filled mark means the report states the commitment explicitly and builds on it; an open mark means it is mentioned or implied but not developed; a dash means it is absent. The scoring is this reviewer's reading. Column keys: MS MathStep, Co Contour, Lo Loom, ML MPL, Mo Mosaic, Mv Motive, Ou Outline, Re Reason, Sp Spine.

Table 8: Design commitments by report.

Commitment	MS	Co	Lo	ML	Mo	Mv	Ou	Re	Sp
Foundations									
1. Lean-hosted layer; no new kernel or foundation	●	●	●	●	●	●	●	●	●
2. Formulas reuse Lean's term language initially	●	●	●	●	●	●	●	●	●
3. Escape hatch to ordinary Lean under the same checks	●	●	●	●	●	●	●	●	●
Unit of authorship									
4. Typed step with a contract as the basic unit	●	●	●	●	●	●	●	●	●
5. Theorem statement frozen before proof search	●	●	●	●	●	●	●	●	●
6. Obligation ledger with ordered dependent telescopes	●	●	●	●	●	●	●	●	●
7. Three synchronised views: argument, obligations, evidence	●	●	●	●	●	●	●	●	○
8. Explicit induction motive and `generalizing`	●	●	●	●	●	●	●	●	●
9. Witness scope; Prop-versus-data distinction for `choose`	●	●	●	●	●	●	●	●	●
10. Deterministic resolution of "this" and other anaphora	○	●	●	●	●	●	●	○	●
11. Distinct `using` / `using only` / preference semantics	●	○	●	○	●	○	●	●	●
Methods									
12. Registered, versioned domain methods with contracts	●	●	●	●	●	●	●	●	●
13. Certified representation views with explicit direction	●	●	●	●	●	●	●	●	●
14. Finite reindexing requires a bijection or multiplicity accounting	○	●	●	●	○	●	●	●	●
15. Locality / derivative-transfer contract developed	●	–	●	●	–	–	○	●	○
16. WLOG as coverage plus transport theorem	●	●	○	●	●	●	●	●	●
17. Analytic moves require a named theorem's exact hypotheses	●	●	○	●	○	●	○	○	●
18. Binder bounds become scoped side-condition facts	●	●	●	●	●	●	●	●	●
Synthesis boundary									
19. Leant as a structural-plumbing service behind an adapter, staged late	●	●	●	●	●	●	●	●	●
20. Result taxonomy; search failure is never refutation	●	●	●	●	●	●	●	●	●
21. Callback receipt is not a kernel certificate (Leant `Verified`)	●	○	●	○	●	○	●	●	●
22. Candidate identity bound to the exact request, not its spelling	●	●	●	●	●	●	●	●	●
23. Multi-criteria ranking; term length is not the objective	●	●	●	●	●	●	●	●	●
24. Shared metavariables solved transactionally	●	●	●	●	○	–	●	○	●

Commitment	MS	Co	Lo	ML	Mo	Mv	Ou	Re	Sp
25. Data synthesis needs a behavioural specification, not just a type	•	•	•	•	•	•	•	•	•
Trust and replay									
26. Discovery mode versus search-free replay; lock or certificate artifact	•	•	•	•	•	•	•	•	•
27. Transitive axiom allowlist, version-aware (Lean 4.29+ computation axioms)	•	•	•	•	•	•	•	•	•
28. Execution isolation of untrusted proof producers	•	•	•	•	•	•	•	•	•
29. A hash identifies evidence; it is not evidence	•	•	•	•	•	•	•	•	•
30. Statement fidelity separate from validity; semantic diff on change	•	•	•	•	•	•	•	•	•
31. Explanatory fidelity as a third notion, with a stricter optional profile	•	•	•	•	•	•	•	•	•
32. Repair after library change is an explicit, reviewable semantic change	•	•	•	•	•	•	•	•	•
Evaluation and roadmap									
33. Refactored-Lean baseline with the same helper lemmas	•	•	•	•	•	•	•	•	•
34. Theorem-leakage control in benchmarks	–	•	•	○	○	•	•	○	•
35. Negative test suite as part of the specification	•	•	•	•	•	•	•	•	•
36. Amortised accounting of adapters and helper libraries	•	•	•	•	•	•	•	•	•
37. Multi-dimensional cost objective, not source length	•	○	•	•	•	•	•	•	•
38. Human comprehension tasks in the evaluation	•	•	•	•	•	•	•	•	•
39. First slice without automation; natural language last	•	•	•	•	•	•	•	•	•
40. Already-short proofs acknowledged as negative controls	•	•	•	•	•	•	•	•	•

Twenty-nine of the forty rows are unanimous. Five more (rows 7, 10, 14, 16, 37) are held by at least seven reports. The six rows with genuine spread (11, 15, 17, 21, 24, 34) concern which method families a report chose to develop, or details of synthesis engineering, not the design of the language. The commitments with spread are also exactly those where a report's file sample determined its emphasis: the locality contract is developed by the four reports that sampled a derivative file, and by no other.

4.2 Nine names for one unit

Every report defines its unit by the same four components: an exact target proposition in an ordered local context; a permitted input policy (which facts and which library); a method or reason that constrains reconstruction; and a checked certificate attached to the step, not to the theorem alone. Contour and Reason write it as an eight-tuple, Mosaic and Spine as a six- or seven-tuple, Loom and MPL as an elaboration judgment; the fields are the same.

4.3 Statement lock, ledger, views, results

The remaining shared machinery can be stated once, as the union of what the nine reports record.

The statement lock (called seal, approval, freeze, or lock) records the elaborated `Expr`, universe parameters, binder telescope, resolved constants and definitions, selected type-class instances, notation provenance, and environment identity. Every report insists it be created *before* proof search and *outside* any untrusted proof producer, and that proof search may choose among proofs

Report	The unit	The central contract, in the report's words (abridged)
MathStep	typed mathematical step	write the mathematical choices; reconstruct the bookkeeping; check the exact claim; preserve the reconstruction
Contour	typed mathematical move	concision belongs in the source; completeness belongs in the certificate
Loom	contracted mathematical step	concision in the document; rigor in the evidence; the correspondence inspectable
MPL	mathematical action with a contract	make intention the unit of authorship, and checked evidence the unit of acceptance
Mosaic	reasoning contract / checkpoint	explanatory compression: retain decisions that carry information, reconstruct forced details
Motive	proof move with a contract	keep the decision; generate its consequences; check every consequence; preserve the exact statement
Outline	semantic checkpoint / certified plan	make structure explicit and plumbing implicit, but retain a checkable account of both
Reason	claim with a contract	write the decisions; state what may be reconstructed; keep the theorem fixed; accept only checked evidence
Spine	proof spine of typed steps	discovery is replaceable; statement identity, scope, and certificate acceptance are not

Table 9: One design, nine vocabularies.

of the locked proposition but never among propositions. Outline gives the sharpest rule: the parser must not decide whether $n - 1$ is natural, integer, or real subtraction by trying all three and keeping the provable one.

The obligation record carries a stable identity, the environment fingerprint, the ordered dependent telescope, the exact target, the permitted-dependency policy, the resource budget, the requested method, the source span, and its dependency edges and scope boundary. All nine reject an unordered bag of pretty-printed hypotheses; all nine forbid a branch-local fact or witness from escaping its scope; all nine require the graph to be acyclic after recursors are treated as ordinary term constructors.

The three views are the author's argument, the generated obligations with status, and the exact Lean evidence, presented as projections of one typed artifact with a source map in both directions. Spine speaks of two layers with expansions but describes the same thing.

The result taxonomy of a reconstruction request, taking the union across reports, is: checked proof; checked negation or checked counterexample at the original statement; fragment-relative negative evidence with its completeness assumptions attached; unsupported structure; budget exhausted; candidate rejected by typing or by policy; statement ambiguous; cancelled. All nine make the same asymmetry explicit: a positive candidate can be re-checked at the exact original goal even if the search abstraction was lossy, whereas a negative result in the abstraction transfers only through a justified adequacy theorem. Every report cites Leant's `Fragment.hs` for this distinction and most cite the `Verified` wrapper's documentation, which states that it records callback acceptance and not a kernel proof. I verified that comment in the local checkout; it reads exactly as the reports say.

Sealing and replay. All nine separate a discovery mode, in which search, retrieval, and model suggestions may run, from a replay or publish mode that loads stored evidence and performs no search. The artifact records toolchain and library revisions, the locked statement, step identities and their evidence, method versions, the transitive axiom inventory, and the source-to-evidence map. All nine state the limits identically: replay is deterministic relative to the pinned environment, a `.olean` is not a cross-version certificate, and a hash locates evidence without constituting it.

Trust policy. All nine demand a transitive *allowlist* of axioms rather than a blacklist of `sorryAx`; six note explicitly that since Lean 4.29 native computation introduces per-invocation axioms, so a policy naming one historical constant is inadequate. All nine want untrusted producers isolated from the checker. This is the one place where the reports' concern maps immediately onto the corpus: the FabiusFunction development uses `native_decide` eleven times.

4.4 Evaluation and roadmap

The evaluation protocols agree on five points: compare against refactored Lean with the same helpers; charge for new adapters and amortise them across a suite; remove the target theorem and its downstream closure from the search environment; ship a negative test suite as part of the specification; and measure author time, reader comprehension on concrete tasks, reconstruction and replay cost, and repair effort after controlled changes, reporting distributions rather than means. Seven reports write down a multi-term cost objective; all nine refuse to claim any measured number.

The roadmaps are also aligned, to the point that one could be substituted for another. Stage one is a Lean-native core with named assertions, calculations, scoped cases, induction with a visible motive, an escape hatch, and *no* automation beyond a small deterministic profile; its exit criterion is that a proof replays with the search service disabled and that deliberately broken variants are rejected. Stage two adds two or three method families drawn from the report's own examples. Stage three connects Leant through a narrow, origin-preserving adapter for structural plumbing only. Stage four adds sealed artifacts, repair, and a hardened validation path. Natural-language surface comes last in every plan, if at all.

4.5 Keywords

Table 10 records the surface vocabulary. The interesting fact is how little it varies and how close it stays to Lean's own tactic names; the variation is a sign of independence, and the closeness a sign that the reports are describing a dialect of Lean rather than a new language.

4.6 Nine soundness arguments, one lemma

Every report proves a relative soundness proposition: if the frozen target is T, every obligation is solved by a term checked at its exact type in its recorded telescope, the graph is acyclic and well-scoped, and the assembled term is rechecked against T under the axiom policy, then the accepted document yields $\Gamma \vdash p : T$. Contour states it as a certificate-assembly theorem; Loom as contract-preserving assembly; MPL, Mosaic, Motive, MathStep as relative proof preservation; Outline, Reason, Spine as soundness of completed elaboration. All nine proofs are the dependent substitution lemma applied in topological order, and all nine correctly add that the argument is a paper argument about a specification, that the final kernel recheck is deliberate redundancy, and that the theorem says nothing about statement fidelity, explanation, or search success. Reason's two corollaries state the useful content most crisply: replacing the search engine by any candidate generator preserves soundness, and a timeout can never establish the goal.

These propositions are correct and, taken together, uninformative. Soundness is guaranteed by the design's most conservative decision, namely to accept nothing the kernel has not checked at the frozen target. The risks the design must actually retire are elsewhere: that the frozen target is not what the author meant, that reconstruction is unpredictably expensive, and that the displayed reason is not the reason the proof works. No report has a theorem about any of these, and all nine say so. The next sections concentrate there.

Construct	Keywords used (report)
universal introduction	`take` (MathStep, Contour, Spine); `fix` (Loom, Mosaic); `given` (Loom, header); "take arbitrary" (MPL)
hypothesis introduction	`assume` (all nine)
local assertion	`have` (MathStep, Contour, Loom, MPL, Mosaic, Motive, Outline); `claim` (Reason); `fact` (Spine)
closing the goal	`show` (MathStep, Loom, Mosaic, Spine); `conclude` (Contour, MPL, Mosaic, Motive, Outline, Reason); `exact` (Loom, Outline)
reduction	`suffices` (all; Mosaic and Outline also `suffice`), always with a bridge obligation
witness use	`choose` (MathStep, Contour, Loom, Motive, Spine); `obtain` (Mosaic, Motive, Outline, Reason); `take witness` (MPL)
case analysis	`cases` (MathStep, Contour, Loom, MPL, Mosaic, Reason, Spine); `split` (Motive, Outline)
induction	`induct ... generalizing` (MathStep, Contour, Loom, Motive, Outline, Reason, Spine); `induction` (Mosaic); `induct on n with invariant` (MPL)
calculation	`calc` (MathStep, Loom, Mosaic, Reason, Spine); `calculate` (Contour, MPL, Motive)
justification	`by method using facts` (all); `from facts by method` (MathStep, MPL, Motive, Reason); `using only` (MathStep, Loom, Mosaic, Outline); `prefer` (Outline); `with budget` (Reason); `plan` (Outline)
escape hatch	`by lean` (MathStep, Mosaic); `lean \{ ... \}` (MPL, Outline, Spine); a Lean block (Contour, Loom, Motive, Reason)
profiles	`proof using [profiles]` (Spine); `with real_ring` (MathStep); notation profile (Contour); `context` block (MPL, Outline)

Table 10: Surface vocabulary across the nine grammars.

5 Where the reports differ

5.1 Genuine divergences

The differences are few and, in each case, adjudicable.

One `choose` or two keywords. MathStep and MPL keep a single `choose` and require the elaborated view to display whether it performed propositional existential elimination, a dependent-pair projection, or classical choice. Loom, Motive, Outline, Reason and Spine use distinct surface forms (`obtain` for elimination, `choose ... := t` for construction). The second is better: a surface form that means different things in `Prop` and `Type` is exactly the ambiguity the design otherwise forbids, and the cost of two keywords is nil. The corpus already uses `obtain` 714 times and `rcases` 725 times.

Notation profiles versus Lean statements. MathStep, Contour and Mosaic sketch notation profiles under which $\sum$, $\binom{n}{k}$, division, and $D[n]$ resolve to fixed Lean terms; Loom, MPL, Motive, Outline, Reason and Spine keep theorem statements in ordinary Lean and add only a proof-block elaborator. The second is the right first step and the first is the right fifth step; the reports that propose profiles also say so. The consequence, discussed in Section 6.3, is that "natural expression" is deferred for statements.

Checkpoint profile versus method-constrained profile. Mosaic distinguishes a profile in which every stated checkpoint must be proved but the proof term need not follow the named method from a stricter profile in which `integrate` must use its declared interface; Reason makes the same split as `by auto` versus a method-constrained annotation; Motive calls the stricter option enforced recipe provenance; Outline distinguishes required plans from preferences.

Contour and Loom lean to the stricter default, MathStep and Spine to the looser. Mosaic's two-profile framing is the right resolution: the stricter profile buys explanatory fidelity at the price of rejecting valid alternative proofs, and that trade should be the author's, per step.

Native worker versus external service. Spine and Reason argue for a project-local Lean worker owning elaborated expressions, with Leant's Haskell engine behind a narrow protocol; Reason correctly credits Leant's own rewrite-analysis document, "Rewriting Leant and Djex in Lean 4," as the precursor. MathStep, Contour, Loom, MPL, Mosaic, Motive and Outline accept an external service initially provided candidates are re-elaborated at the exact goal. These are compatible: all nine put Leant at stage three or later.

How early to harden. MPL, Motive, Mosaic and Outline specify sandboxing, a separate validation process, and a trusted challenge statement as part of the design; Contour, Loom and Spine describe them as a later deployment mode. For a research prototype used by its own author the later position is right; the earlier position becomes right the day model-generated proofs are accepted from outside.

What `using only` restricts. Mosaic restricts local premises but not the library unless a separate `library only` clause is given; Contour splits a mathematical input policy from a foundation policy; Spine permits named facts, their typing vocabulary, and a profile library. All three are the same design once written down, and it is the design to adopt: two independent restrictions, local facts and library inventory, with the transitive axiom audit as a third.

Analytic ambition. Motive (dominated convergence, with the counterexample $f_n = n \cdot \mathbf{1}_{(0,1/n)}$), Mosaic (integral comparison with orientation and integrability obligations) and Spine (limits along a filter, with nontriviality) develop analytic moves; Contour and MathStep counsel that analytic moves be conservative and demand named theorems at first. Both are right; the audit's finding that analysis plumbing is 7% of the corpus says the conservative version should nevertheless be built early.

5.2 Distinctive contributions

Table 11 lists ideas that appear in one or two reports only, with a short assessment. They are the "explore" part of this review: none is part of the consensus, and several deserve to be.

Table 11: Ideas particular to one or two reports.

Idea	Report	Assessment
`remaining =>` rebuilds all uncovered constructor cases from induction hypotheses, with a sealed coverage certificate invalidated when a constructor is added	MathStep	Strong and cheap: it is `cases` plus a recorded constructor set. The natural-deduction example (eleven constructors, one interesting) is exactly the use case.
Reader-facing aliases are lexical and versioned; "dyadic sample formula" resolves to a pinned declaration and instantiation, never to a search	MathStep	Should be in the consensus. It is also how the corpus already works: the long names are the certificate identity.
Certificate-assembly theorem with an explicit treatment of shared metavariables solved transactionally	Contour	The sharpest statement of the scheduler invariant; Aesop's handling of shared metavariables is the existing precedent, which Spine notes.
Representation views are relations $V(a, b)$ with a directed transport theorem $V(a, b) \to P(a) \to Q(b)$, bounded search depth, and a recorded route	Contour	The best formulation of "views"; composition is still unspecified (Section 6.2).

Idea	Report	Assessment
"Similarly" means instantiate a checked template and re-check; the strict Lambert bound exposes the new $w > 0$ obligation	Contour, Motive	Correct and implementable as a macro over the core.
A hierarchy of inference risk from definitional reduction up to mathematical choice, with silence permitted only at the bottom	Loom	A good user-interface principle; it maps onto the audit's routine portfolio.
Conditional proof skeleton $K : \prod_{h_1} \prod_{h_2} A$ checked before holes are solved	Loom, MPL	Useful for testing recipes without `sorry`; also what a typed sketch is.
Six verification states for a candidate, from proposed to specification-proved, as a table	Loom	The clearest of the taxonomies; adopt.
Learn reusable recipes from repeated accepted patterns, installed only with a checked theorem	Loom, Spine	Promising; the audit's bigram data (`have` then `rw` 5,486 times; `rw` then `ring` 1,283 times) is where mining would start.
Effect kinds for a synthesis request: proof-only, data-producing, statement-resolving, the last needing review	MPL	Should be in the consensus; Spine's "four kinds of holes" is the same idea.
`work in B via f` needs reflection for equality, one-way preservation for implication, order reflection for inequalities	MPL	Precise and correct; the corpus's 899 `Nat.cast_*` references make it the most-used view.
A "semantic choices" panel listing formal-vs-analytic series, total-vs-restricted inverse, natural-vs-integer subtraction	MPL	The seed of the statement lint proposed in Section 7.4.
A false but unused checkpoint must fail the document even if the theorem is provable another way; a true unused one is marked, not rejected	Mosaic, Spine	Important and easy to get wrong; adopt as a test.
Witness contracts for "sufficiently small" ($\delta = \min$, with positivity and two comparisons)	Mosaic	A good example of a witness method whose synthesised value is uninteresting and whose obligations are not.
Constructive and classical modes affect provider selection and the final audit; a decidable case split is not classical	Mosaic	Correct; interacts with the allowlist.
Comparison with the June 2026 Visored preprint, which emits Lean optionally	Mosaic	Useful positioning: the consensus's non-negotiable is mandatory Lean acceptance.
Reasons are refinable in place: a failed `by monotonicity` becomes `by monotonicity of log on positive_reals`	Motive	A small interaction design that matters; adopt.
A context index organised by mathematical role (membership, positivity, nonzeroness, regularity) with provenance, scoped to cases	Motive, Outline	Sensible engineering; it is what `positivity` and `omega` already consume implicitly.
The Richardson normalisation control shows the exact condition $\forall r < n,\ q^{r+1} \neq 1$ rather than the tempting $q \neq 1$	Motive	The best single example that "repair by strengthening" is a theorem change.

Idea	Report	Assessment
Plan / prefer / `using only` as three distinct guidance forms	Outline	Should be in the consensus.
The residual-radius theorem re-proved by bracketing both endpoints, avoiding the sign split entirely	Outline	A reminder that some verbosity is a proof choice, not a language deficit; a prototype should show this in ordinary Lean first.
A five-file artifact bundle (`.outline`, `.expanded.lean`, `.lock.json`, `.map.json`, `.audit.json`)	Outline	Concrete and reasonable; the expanded Lean file is the right interoperability format.
Citing the 2026 benchmark-faults preprint for specification defects in Lean benchmarks	Outline	Reinforces the leakage and statement-review points.
Explore / resolve / publish as three modes, with an inconsistent context flagged rather than exploited	Reason	Adopt; the third mode is the sealed mode of the others.
Privacy of remote services as a policy separate from evidence	Reason	Correct and otherwise unmentioned.
An executable toy checker (simply typed with products) whose 32 tests exercise origin binding, forged certificates, and zero budgets	Reason	The only executable artifact among the nine; modest, and honest about being so.
Local instances installed from proved facts (`letI`) as an evidence mechanism, distinct from structural instances that change meaning	Spine	Correct and measurable: 162 `letI`/`haveI` uses in the corpus.
A surviving-support method for coefficient extraction through a homomorphism	Spine	Narrow but reusable; typical of the EGF files with the highest bridge density.
An iteration contract ($F(x) \preceq F(T^n x)$ from a one-step comparison and an orbit identity)	Spine	A clean example of a method whose contract is a small theorem schema.

6 Assessment

6.1 What is strongest

Ranked by the ratio of value to implementation risk, the strongest ideas are the ones every report shares, in this order.

1. **The statement lock.** It costs one elaboration pass and a recorded `Expr`; it retires the single most dangerous failure mode of proof-driven interpretation; it needs no automation to be useful. It is also independently motivated by the benchmark-integrity concerns Outline cites.
2. **The obligation ledger with scoped telescopes and roles.** Lean already has the goals; what is missing is persistence, a source map, and per-obligation policy. The audit shows the demand: 2,243 one-line obligations and 32% of invocations in the reconstructible families.
3. **Search-free replay with a transitive axiom allowlist.** Every ingredient exists (`says`, `#print axioms`, `.olean`s, pinned toolchains); the design contribution is to make the step-level map and the allowlist mandatory at publication.
4. **Four method families with contracts**: ordered-field side conditions, finite-sum index management, cast and scalar transport, and local derivative/limit plumbing. Each is confirmed by the audit to be a substantial share of the corpus, and each has an existing Mathlib tactic or lemma family to wrap (Section 7.2).
5. **The negative test suite as specification.** The nine suites, deduplicated in Section 7.5, are the most concrete artifact the reports produced, and they are executable before any language

exists: each is a mutation of an existing ProveIt proof.

6. **The evaluation discipline**: refactored baselines, leakage control, amortised accounting. This is worth more than it appears, because it is what will prevent the project from fooling itself with the four or five examples it was designed around.

6.2 What is weak, over-engineered, or under-specified

- **View composition is unspecified.** All nine want certified representation views; CONTOUR bounds search depth and records the route; MOSAIC lists "certified view composition" as future research. None says what happens when three views compose, whose side conditions are collected in which order, or how the normal form chosen by one view interacts with the next. In the corpus this is the dominant bridge (899 `Nat.cast_*` references, 598 `algebraMap`), so it cannot be left to a later stage. Mathlib's `norm_cast` simp-set discipline (`@[norm_cast]` lemmas classified as elim, move, and squash) is the existing answer and should be the starting point.
- **Explanatory fidelity is named, not solved.** All nine distinguish "the term proves the proposition" from "the displayed reason is why." The only enforceable proposal is MOSAIC's method-constrained profile plus OUTLINE's plan-conformance check, which certify that the named method's certificate tree has the advertised root. That is worth building. Anything stronger, such as certifying that a lemma was indispensable, is correctly abandoned by every report.
- **Contract explosion.** The nine reports name about forty methods (Appendix B); each needs a contract, obligations, negative tests, documentation, and a version. The reports acknowledge the maintenance cost but do not bound it. Section 7.1 argues the cost collapses if a contract is an annotated theorem type rather than a new object.
- **The soundness theorems are ceremony.** See Section 4: nine restatements of the substitution lemma, each with a page of caveats. A single paragraph would have served, and the space would have been better spent on the unsolved problems.
- **Trust engineering is premature for the prototype.** A sandboxed second checker with a trusted challenge statement is the right design for accepting proofs from strangers and the wrong first milestone for a language whose first user is its author.
- **The evaluation plans are plans.** Every report proposes measurements; none performed any, and all nine say so. Section 7.3 proposes the one measurement that can be done now.
- **Nine names.** The reports spend real effort on naming. The name does not matter; what matters is that the surface stays a dialect of Lean (Table 10 shows it already does).

6.3 Blind spots common to all nine

- **Definitions.** All nine design a *proof* language and keep definitions and statements in Lean. But much of the friction the reports themselves document is definitional: a bounded function as a subtype in `Basic.lean`, a nonnegative-real Lipschitz parameter, `Finset.range (n+1)` versus `Icc 0 n`, a formal series versus an analytic one. A proof language cannot repair a definition chosen for the library's convenience. Section 7.6 argues that controlled-language *definitions*, with their coercion and totality conventions made explicit, are the larger lever for "natural expression."
- **Instances and universes.** Only SPINE discusses local instances, and no report discusses instance diamonds, `outParam` failures, universe constraints, or the elaboration order problems that experienced Lean authors spend real time on. The corpus's 162 `letI`/`haveI` lines are the visible tip.
- **Elaboration performance.** No report treats slow elaboration or heartbeat limits as a shortcoming of Lean. The corpus raises `maxHeartbeats` in fourteen FabiusFunction files and on nearly ten thousand lines repository-wide. A step layer that re-elaborates each obligation

could make this worse; the reports' insistence on bounded budgets per obligation is the right instinct, but none connects it to the existing problem.
- **The reader who edits.** Maintenance is discussed as replay and repair, never as collaboration: who owns a method contract, how two authors' profiles compose, how a change to a domain pack propagates through a thousand files.
- **Migration by decompilation.** Only REASON raises recovering a chain of claims from an existing Lean proof. Given that the corpus is already 32% assertions, lifting existing proofs into the step layer is both the cheapest adoption path and the best test corpus, and it deserves to be a stage in the roadmap.
- **The existing supply.** Discussed next.

7 Additions: what the nine reports miss

7.1 A contract is an annotated theorem type

The reports describe a method as a registered object with an applicability pattern, an obligation generator, a reconstruction procedure, an explanation renderer, and a version. Building forty of these is the cost the reports underestimate. But look at what the contracts actually are. LOOM's `derivative_from_local_identity` recipe requires $x \in s$, $s \in \mathcal{N}(x)$, $f = g$ on s, and `HasDerivAt g v x`, and ensures `HasDerivAt f v x`: that is the statement of `HasDerivAt.congr_of_eventuallyEq` composed with `Filter.eventuallyEq_of_mem` and `IsOpen.mem_nhds`. CONTOUR's `multiply(c)` requires $0 \leq c$ and $a \leq b$ and produces $ca \leq cb$: that is `mul_le_mul_of_nonneg_left`, which the corpus applies 254 times. MOTIVE's `reindex` contract is the hypothesis list of `Finset.sum_bij` and its `to_additive`-generated variants. MPL's `work in B via f` with reflection is `Nat.cast_injective` plus `push_cast`.

The proposal, then, is this: *a method is a theorem with role annotations on its hypotheses.* Concretely, an attribute of the form

```
@[method (name := reindex) (roles := [input, input, side, side, side, side])]
theorem Finset.sum_bij ...      -- the contract IS the type
```

declares which hypotheses are mathematical inputs the author supplies (the index map, the summand equality) and which are side conditions to be discharged by the routine profile (membership, injectivity, surjectivity). A step `have h : P by reindex using facts` elaborates to `refine Finset.sum_bij ?i1 ?i2 ?s1 ?s2 ...`, unifies the conclusion with P (this is what `apply` already does), fills the `input` holes from the named facts, and records every remaining hole as a ledger entry tagged with its role and the profile that may discharge it. The explanation renderer is a docstring template over the theorem's binder names.

The consequences are large. There is no new trusted code and no method language: applicability is unification, the obligation generator is the theorem's telescope, and reconstruction is application. A domain pack is a collection of annotated theorems, which is also what MATHSTEP means by "support lemmas should be ordinary library assets" and what LOOM means when it says a recipe should expose the same result as a reusable theorem; the difference is that here the theorem is the *only* representation. The "forty methods" become forty attributes on existing Mathlib and ProveIt declarations, plus a dozen composite lemmas (locality-then-chain-rule is one `theorem`). Versioning is Mathlib's. Negative tests are theorems whose hypotheses cannot be discharged. What remains genuinely new is the ledger, the seal, and the roles.

Two things do not fit this mould and should be recognised as such: *normalisers* (`ring`, `field_simp`, `push_cast`, `omega`), which are proof-producing procedures rather than theorems and are consumed as side-condition dischargers; and *structural plans* (induction with a motive, WLOG, case coverage),

which are eliminators plus bookkeeping. Both already exist in Lean; the layer wraps them.

7.2 Lean already has most of the supply; what is missing is the accounting

Table 12 maps the reports' proposed constructs to what exists in Lean 4.32 and Mathlib at the commit ProveIt pins. I verified each file's presence in the local Mathlib checkout.

Proposed construct	Existing mechanism (Lean 4.32 / Mathlib)	What is actually missing
Statement lock / seal	the header is elaborated before the body	a recorded, diffable artifact; a policy that search cannot touch it
Obligation ledger	goal list; `?_` holes; InfoTree	persistence, roles, source map, per-obligation budget and policy
Sealed replay of a searched step	`says` (`simp? [X] says simp only [...]`); `exact?` to `exact`; `polyrith` to `linear_combination`; `.olean`	step-level lock file; mandatory at publication; corpus uses none of these
Axiom allowlist	`#print axioms`; `Lean.collectAxioms`; per-invocation `native_decide` axioms	enforcement as a build gate with a transitive allowlist
Multiply an inequality by a nonneg factor	`gcongr` with `positivity` side goals	a ledger entry for the side goal
Side conditions	`positivity`, `omega`, `linarith`, `nlinarith`, `bound`, `norm_num`	the routine profile as an explicit, versioned object
Cast and scalar transport	`push_cast`, `norm_cast`, `exact_mod_cast`, `zify`, `qify`, guarded `Nat.cast_sub`	guard obligations surfaced with provenance; view direction recorded
Finite reindexing	`Finset.sum_bij`, `sum_nbij'`, `sum_equiv`, `sum_image`, `sum_range_reflect`	role annotations; multiplicity diagnostics
Locality and derivative transfer	`Filter.EventuallyEq.deriv_eq`, `HasDerivAt.congr_of_eventuallyEq`, `IsOpen.mem_nhds`, `filter_upwards`	one composite theorem plus a contract
Continuity / differentiability side goals	`fun_prop`, `continuity`	nothing but the ledger entry
Without loss of generality	Mathlib's `wlog` tactic (`Mathlib/Tactic/WLOG.lean`)	coverage and transport as displayed obligations
Mixed relation chains	`calc` with `Trans` instances	already exactly the "registered transitivity" the reports ask for
Reduction with a bridge	`suffices h : Q by ...`	nothing
Unused checkpoints	`linter.unusedTactic`, `haveLet` linter	an assertion-level linter and plan-conformance check
Discovery suggestions	`exact?`, `apply?`, `simp?`, `aesop?`	none of these are used in the corpus

Table 12: Proposed constructs against existing mechanisms.

The lesson is not that the reports are redundant. It is that they mis-describe the deliverable. Every one of them presents the language as a way to *obtain* routine consequences; the audit and this table say the consequences are already obtainable, in the sense that a Lean author who knows `gcongr`, `positivity`, `push_cast` and `filter_upwards` can already write the step in one line. What is missing is that the one line leaves no trace: no record of which side conditions were discharged and how, no separation of the mathematical input from the routine, no seal that lets

the step be replayed without re-running `simp`, and no explanation. The design is an *accounting layer* over existing automation. Framing it that way shrinks the project, removes the reason to build a new prover, and makes the first milestone concrete: make `says` the default for every automated step and make its output land in a ledger.

7.3 Measure demand before building supply

The audit in Section 3 is a crude instrument and it already changed one priority (analysis over reindexing). A better instrument is cheap. A Lean metaprogram that walks the InfoTrees of a compiled file can record, for every tactic node, the goal before and after, the declarations the produced term references, and whether the node is a `have` whose sub-proof is closed by a decision procedure. From this one obtains, without heuristics: the fraction of goals closed by each tactic; the distribution of "bridge chains" (maximal runs of CAST/INDEX/REPR nodes between two assertions); the most frequent lemma bigrams; and, most usefully, for each candidate method contract, the number of corpus sites where its conclusion unifies with a goal and its side conditions are discharged by the routine profile. That last number is the expected yield of each method before any is built.

The audit also suggests two metrics to track once a prototype exists. *Bridge density* is the reconstructible share per file, which ranges from 10% to 71% today and should fall in migrated files without the assertion share falling. *Assertion coverage* is the fraction of tactic invocations that belong to a named assertion's sub-proof rather than to the top-level script, which measures how much of a proof is already spine-shaped.

7.4 A statement lint and a controlled back-rendering

All nine reports leave statement fidelity to "human review of the elaborated statement," with MPL's semantic-choices panel and Spine's expanded header as aids. This can be made mechanical in the common cases. A deterministic lint over the locked `Expr` should flag the constructions that the reports' own counterexamples turn on:

- natural-number subtraction or division anywhere in the statement, with the hint "truncated; did you mean the integer or rational operation?";
- real division, `Real.sqrt`, `Real.log`, or an inverse, applied to an argument not accompanied by a nonzero, nonnegativity, or positivity hypothesis in the same statement;
- `deriv` or `iteratedDeriv` without a `HasDerivAt`, `Differentiable`, or `ContDiff` hypothesis on the same function;
- `Finset.range` and `Icc`/`Ico` mixed in one statement, or a range bound that differs from the mathematical one by one;
- a $\forall \varepsilon\, \exists \delta$ block whose δ is bound outside the ε it should depend on;
- a field, integral-domain, or characteristic-zero instance on a ring that the surrounding development treats as general;
- `Classical.choice` reachable from a statement in a development that declares a constructive profile;
- formal-series composition or evaluation where an analytic function is expected, or vice versa.

Each rule is a syntactic pattern over the elaborated term with a fixed message; none can be fooled by proof search because it runs on the lock. In addition, the lock should be rendered back into controlled English by a deterministic printer in which the flagged constructions are spelled out ("the truncated difference $n \dot{-} 1$"; "the total real division a/b, which is 0 when $b = 0$"). Verbose Lean's rendering machinery is a precedent. The author confirms the rendering, not the `Expr`. This is the cheapest available answer to the only unsolved trust problem the reports identify.

7.5 The ledger as a task format, and the consolidated adversarial suite

A ledger entry is a closed, typed, budgeted subgoal with an explicit environment, an axiom policy, and a dependency exclusion list. That is precisely the input format an automated prover, agentic or otherwise, needs, and precisely the format a leakage-controlled benchmark needs. Two consequences follow. First, the step layer should treat a language model or an external prover as one more provider behind the same interface, which all nine reports say; but it should also *export* its ledger as a benchmark, so that ProveIt's own obligations become a held-out test set with the frozen statements, the allowlist, and the pre-target environment already attached. Second, the nine reports' negative test suites, once deduplicated, are the seed of an adversarial obligation suite that can be run against any provider today, because each test is a mutation of an existing ProveIt proof. Table 13 gives the deduplicated list.

Table 13: Consolidated adversarial suite, deduplicated across the nine reports.

#	Mutation	Reports	Required behaviour
1	Insufficient induction motive; hypothesis needed at a changed argument	9	propose `generalizing` visibly; never grant the stronger hypothesis silently
2	Existential witness escapes its scope, or $\forall\exists$ is read as $\exists\forall$	9	reject the scope violation; never reorder quantifiers
3	Fact from one case used in a sibling case, including via a cache	9	reject the scope map or the term
4	Bounded search exhausted	9	report exhaustion or unsupported structure, never refutation
5	`sorryAx` or an unapproved axiom reachable transitively	9	fail the allowlist even when elaboration succeeds
6	Definition or instance changed while the printed statement is unchanged	9	invalidate stale evidence; compare elaborated statements
7	Type-correct data term with the wrong behaviour ($A \to A \to A$ projection; state monad)	9	require the behavioural specification; expose the data choice
8	Natural subtraction transported without $b \le a$ ($m = 0$, $n = 1$)	9	generate the guard or stop the transport
9	Cancel a factor without nonzero or unit evidence ($\mathbb{Z}/6$: $2 \cdot 0 = 2 \cdot 3$; $2s_1 = 1$ over $\mathbb{Z}$)	8	leave the obligation; do not import a field-only theorem
10	Reindex a sum through a non-injective map	7	refuse the bijection plan or demand multiplicity accounting
11	WLOG with an asymmetric hypothesis ($x = 0$; $x = \lvert x\rvert$ "by symmetry")	7	demand context transport; name the untransported hypothesis
12	Mutually justified claims forming a cycle	7	keep both open
13	Repair by strengthening ($\mathbb{Q}$-algebra to field; add openness; $\lvert q\rvert < 1$ to $\le$)	6	present as a statement change, never as progress on the old target
14	Finite-sum rule applied to an infinite or conditionally convergent sum	5	require the summability theorem
15	Differentiate from equality at one point (t versus 0 at 0)	4	require neighbourhood equality
16	Natural division transported without divisibility (3/2)	4	refuse the exact-division bridge
17	Theorem leakage through an alias or downstream lemma	4	mark the run invalid under its dependency policy
18	Formal-series composition without `HasSubst`, or a formal identity read analytically	4	generate the admissibility or convergence obligation
19	Limit interchange from pointwise convergence alone ($n\mathbf{1}_{(0,1/n)}$)	3	require the chosen theorem's domination hypotheses

#	Mutation	Reports	Required behaviour
20	False but unused intermediate assertion in a trivially provable theorem	3	reject the document
21	Strict inequality multiplied by a merely nonnegative factor	3	require positivity or a zero case

7.6 Definitions before proofs

The brief asked for a language in which mathematicians "express their ideas" more naturally. All nine reports answered for proofs and deliberately left statements and definitions in Lean; this was the right scoping for a first design, and the reports say so. But the corpus suggests that the larger and less-explored gain is on the definition side. A definition fixes the representation that every later proof must bridge: whether a bounded function is a subtype or a function with a hypothesis, whether a sum runs over `range (n+1)` or `Icc 0 n`, whether a coefficient lives in $\mathbb{Q}$ or in an algebra over it, whether a derivative is a hypothesis or a `deriv` expression. The 4.0% CAST and 4.8% INDEX shares in the corpus are largely the interest paid on such choices.

A controlled-language layer for *definitions* would let an author write the mathematical object once, with explicit answers to the questions the statement lint asks (totality conventions, coercion targets, index conventions, regularity assumptions), and would generate the Lean definition together with the bridge lemmas that later proofs need: the cast lemmas, the interval conversions, the `simp` normal form. This is a larger design problem than the proof layer and none of the nine reports touches it. It should be on the research agenda beside the proof language, because it attacks the source of the bookkeeping rather than its symptoms.

7.7 What the convergence does and does not show

Eight of nine reports come from the same model family and all nine answered the same brief on the same day. Their agreement therefore does not establish that nine independent designers would reach this architecture. What it does establish is stronger than it looks. The samples were nearly disjoint, so the *friction families* were found in different files and are unlikely to be artefacts of any one example. The negative controls were found by each report against its own thesis. The counterexamples and alternative proofs coincide across reports that could not have seen each other. And the architecture the reports converge on is, as Section 7.2 shows, largely a description of what Lean and Mathlib already do plus a small, well-defined accounting layer. A design that a model family reaches nine times from nine different evidence samples, and that turns out to be an incremental extension of existing practice, is a design with a low chance of being wrong in its outline. Its unknowns, which the reports identify honestly, are all empirical: cost, fidelity, and whether authors find the result better to write and read.

8 A consolidated recommendation

The nine roadmaps, the audit, and the additions above combine into the following plan. It is a recommendation with gates, not a schedule.

1. **Instrument before building.** Run the InfoTree audit of Section 7.3 on the FabiusFunction development. Publish bridge density and assertion coverage per file, the routine-profile portfolio, and the expected yield of each candidate method. Gate: the four method families are ranked by measured yield.
2. **The accounting core.** A Lean package providing: a step block with `have`, `obtain`, `suffices`, `calc`, `cases`, and `induct ... generalizing`, plus a `lean` escape; a statement lock recorded as an artifact; a ledger with roles, budgets, and a source map; `says`-style sealing of every automated step; an axiom allowlist enforced at build; the statement lint of Section 7.4. No

automation beyond the measured routine profile. Gate: ten migrated ProveIt theorems replay with all search disabled, and the twenty-one mutations of Table 13 that apply to the core are rejected with the required diagnostics.

3. **Methods as annotated theorems.** Add the `@[method]` attribute of Section 7.1 and annotate the theorems behind the four families, starting with the ones the audit ranks highest; write the composite theorems (locality plus chain rule; guarded cast transport; bijective reindexing with membership) as ordinary lemmas. Gate: the twenty-three sampled files migrate with bridge density at least halved and assertion share unchanged, against a refactored-Lean baseline that has the same composite lemmas.
4. **Migration by lifting.** Build the decompiler REASON hints at: lift an existing `by` block into a step block by recognising `have` chains and method-shaped tactic runs. Gate: half of the corpus lifts automatically with no change to the elaborated theorem.
5. **Synthesis behind the ledger.** Connect Leant for structural plumbing, then other providers, through the origin-preserving adapter all nine describe; export the ledger as a leakage-controlled benchmark. Gate: providers are interchangeable without any change to acceptance, and negative outcomes carry their labels to the interface.
6. **Only then**: notation profiles, controlled-language definitions, and a natural-language editing assistant whose output is a patch to the source.

9 Conclusion

The nine reports agree on the diagnosis (granularity), on the cure (a typed step with a contract, a frozen statement, a scoped ledger, certified views, an honest synthesis boundary, and search-free replay), on the guardrails (allowlists, negative tests, refactored baselines), and on the order of construction. The corpus agrees with them: a third of tactic invocations are bookkeeping, uniformly across proof lengths, and the proofs are already organised as chains of assertions waiting to be made primary. Where this review departs from the reports is in what the work consists of. Almost every method the reports name is already a Mathlib tactic or lemma; contracts can be types rather than a new language; the seal already exists in miniature as `says`; the routine profile is an empirical fact, not a design choice; and statement fidelity can be attacked mechanically with a lint over the lock. The deliverable is smaller than the reports describe, and better defined: an accounting layer that makes existing automation leave a trace. The larger open problem, which none of the reports takes up, is on the other side of the colon, in the definitions.

A Report cards

Each card states the report's unit, its sample, what it contributes that the others do not, and its principal weakness. Assessments are this reviewer's.

MathStep (`docs/ideas/MathStep`, 42 pages)

Unit: typed mathematical step; three contracts (meaning, evidence, reconstruction). *Sample:* algebraic branch of a root, autonomous iterated derivatives, dyadic substitution under a sum, natural-deduction `mapAxioms` with eleven constructors. *Distinctive:* `remaining =>` constructor rebuild with a coverage certificate; lexical, versioned aliases separating reader-facing names from certificate identity; a six-component cost vector; explanation-preserving proof compression as a research direction; the observation that dropping `hq` after the square identity changes the theorem, not the proof. *Weakness:* the longest report, with the most material duplicated from Lean's own validation documentation.

Contour (`docs/ideas/contour-proof-language`, 38 pages)

Unit: typed mathematical move with fixed conclusion, dependency context, method contract, certificate. *Sample:* binomial inversion, Lambert bounds, Lagrange-inversion uniqueness, Pascal-tail induction. *Distinctive:* the certificate-assembly theorem with transactional shared metavariables; views as directed transport relations with bounded route search; the strict-Lambert stress test for "similarly"; forward and backward frontiers in the editor; a module-boundary table listing what each component must *not* decide; a shipped EBNF. *Weakness:* method names (`multiply`, `reindex`, `coefficients`) are presented as new interfaces where they are existing lemmas.

Loom (`docs/ideas/loom_proof_language`, 30 pages)

Unit: contracted mathematical step; recipes. *Sample:* autonomous derivatives, Abel polynomials over a $\mathbb{Q}$-algebra, Boolean-cube expansion. *Distinctive:* the hierarchy of inference risk; the conditional proof skeleton; six verification states as a table; the ranking objective $J(r)$; four correctness questions (typing, statement, contract, explanation); recipe learning from repeated patterns; the EGF interface. *Weakness:* the least concrete implementation plan; the recipe syntax is closest to a new language rather than a Lean dialect.

MPL (`docs/ideas/mathematical_proof_language`, 35 pages)

Unit: mathematical action with a logical and an explanatory contract. *Sample:* Abel coefficients, autonomous derivatives, associated Stirling recurrence, weighted scaling. *Distinctive:* effect kinds (proof-only, data-producing, statement-resolving); the reflection/preservation distinction for `work in B via f`; the shifted-EGF theorem as a reusable coefficient rule; suggestions as transactions; the semantic-choices panel; the scaling proof as a control against over-engineering. *Weakness:* the surface syntax is the most prose-like and hence the least precisely specified.

Mosaic (`docs/ideas/mosaic_proof_language`, 32 pages)

Unit: reasoning contract; proof spine, obligation graph, certificates. *Sample:* factorial-strength flatness bound, sharp Lipschitz constant, floor-square-root sum. *Distinctive:* explanatory compression as the objective; false-but-unused checkpoints must fail; checkpoint versus method-constrained profiles; witness contracts for "sufficiently small"; constructive and classical modes; the five-configuration evaluation; positioning against Visored. *Weakness:* `integrate` and `count` are the most ambitious methods proposed and the least specified.

Motive (`docs/ideas/motive_proof_language_article`, 30 pages)

Unit: proof move with a contract; landmarks versus routine deductions. *Sample:* binomial inversion, Lambert bounds, geometric Richardson polynomials. *Distinctive:* reasons as refinable library interfaces; the context index by mathematical role; the dominated-convergence counterexample; the Richardson control with the exact normalisation condition; the ten-node reindexing ledger; sixteen conformance tests. *Weakness:* the smallest sample, with two of three files shared with Contour.

Outline (`docs/ideas/outline_proof_language`, 34 pages)

Unit: semantic checkpoint and certified proof plan. *Sample:* Basic, MeanValueBracket, QPascalSummation, Arithmetic, FloorSqrtSum. *Distinctive:* plan/prefer/`using only`; the residual-radius theorem re-proved without a sign split; the abstract WLOG proposition; bridge suggestion; the five-file artifact bundle; the benchmark-faults citation; proof planning and Apply2Isar as lineage; the noncommutative-semiring caution for Gaussian coefficients. *Weakness:* the taxonomy table conflates categories that the other reports keep apart.

Reason (`docs/ideas/reason_proof_language`, 32 pages)

Unit: claim with a reconstruction contract (an eight-tuple). *Sample:* Basic, Arithmetic, AffineDifferenceOrbit, FloorSqrtSum. *Distinctive:* the fold-through-absolute-value view; explore/resolve/publish modes; search-independence and safe-failure corollaries; inconsistent contexts flagged; privacy of remote services; worker generations and stale replies; claims as cache and repair units; the bidirectional (decompilation) problem; the only executable artifact (a 32-test toy checker). *Weakness:* no pinned commit for either repository.

Spine (`docs/ideas/Spine_Proof_Language`, 39 pages)

Unit: proof spine of typed steps with an authored and a generated layer. *Sample:* Thue–Morse prefixes, characteristic-function uniqueness under an affine recurrence, algebraic inverse germs. *Distinctive:* four kinds of holes; local instances as evidence; the iteration and surviving-support contracts; the hockey-stick alternative proof classified as a different plan; the vanishing-along-a-contracting-orbit proposition as a library improvement to be charged to the baseline; existence, selection, and computation of a root as three operations. *Weakness:* it is the only report whose sample contains no file shared with another, so its friction families rest on its own evidence alone.

B Method inventory

The method names proposed across the nine reports, grouped into families. Names are as written in the reports; a name in parentheses marks the report.

Ordered-field algebra and side conditions

`algebra`, `order`, `rational_arithmetic`, `positive_square_root` (MathStep); `multiply(c)`, `divide(c)`, `normalize`, `arithmetic`, `algebra using` (Contour); `normalize using rational_algebra(A)` (Loom); `rational_scalar_algebra` (MPL); `algebra in Real`, `order using`, `bound` (Mosaic); `multiply tangent by`, `real_algebra`, `cancellation` (Motive); `field_arithmetic`, `interval_arithmetic`, `exact_natural_arithmetic` (Outline); `ordered_arithmetic`, `semiring`, `scalar_normalization`, `additive_normalization` (Reason); `algebra`, `arithmetic` (Spine).

Finite sums and indices

`substitute L inside E under binder` (MathStep); `reindex(k = j+i)`, `sum_algebra`, `split_last`, `finite_sum_normalize`, `finite_sum_support` (Contour); `reindex T by U`, `split target_sum by` (Loom); `reindex by e` (MPL); `count fibers` (Mosaic); `reindex k := j+i`, `alternating_row` (Motive); `reindex via`, `split the left sum at`, `count pairs ... first by ... then by` (Outline); `partition target subsets by membership`, `partition_product parity`, `count D by` (Reason); `finite_sum(split_last, combine)`, `surviving_terms`, `retain only` (Spine).

Casts and representation transport

`guarded_cast_normalization`, `normalize casts` (MathStep); `coefficients`, cast-transport view (Contour); typed rational-scalar normaliser (Loom); `work in B via f`, `faithful transport`, `conclude in Nat by injectivity` (MPL); `transport goal to Rational preserving Nat arithmetic` (Mosaic); homomorphism transport certificate (Motive); `calculate over Rat` (Outline); `view nat_to_rat` (Reason); guarded arithmetic views, `coefficient_homomorphisms` (Spine).

Local analysis

`differentiate ... at x using`, `locality`, `chain_rule` (MathStep); `locally at x on s`, `differentiate ih`, `derivative_from_local_identity` (Loom); `differentiate IH locally at x on U` (MPL); `integrate dom over [0,x]`, `derivative_bound`, `derivative_of_lipschitz` (Mosaic); `lower_slope on [a,b]`, `intermediate_value`,

`derivative_bounds on` (Outline); `restrict_to_neighborhood`, `differentiate local_identity at z`, `affine_orbit_derivative` (Reason); `continuity`, `geometric_decay`, `limit_order`, `iterate` (Spine); dominated-convergence move (Motive).

Formal series

`associativity` with `HasSubst`, `substitution_algebra` (Contour); `Lagrange_inversion`, `formal_exponential_rules` (Loom); `Lagrange_Burmann`, `formal_exponential_algebra`, `exponential_coefficients`, `compare exponential coefficients` (MPL); `formal_implicit_root`, `positive_power_constant_term` (Spine).

Structural and logical

`remaining =>` (MathStep); `routine`, `assemble`, `apply(thm)` (Contour); `by routine`, `search` (Loom); `conclude R from h through Q` (MPL); `routine` with `from only` (Mosaic); `structural`, `by auto` (Reason); `plan`, `search`, `prefer` (Outline); `lean \{\}` escape (all).

Symmetry and reduction

`wlog` schema (MathStep, Contour, MPL, Motive, Spine); `reduce by symmetry x -> -x to 0 <= x` (Mosaic); `by symmetry swap(x,y), assume x <= y` (Outline); `symmetry abs_neg`, `wlog x in S using reduction` (Reason); `similarly` as template instantiation (Contour, Motive).

C Audit method and reproducibility

The audit script is `docs/unified_report/tools/audit_proveit.py`; its JSON output and printed summary are beside it. It was run on the local ProveIt checkout at commit `7c4e3f10` over `Analysis/FabiusFunction/Lean/FabiusFunction` with the twenty-three sampled files as a comparison group:

```
python audit_proveit.py <ProveIt>/Analysis/FabiusFunction/Lean/FabiusFunction \
    --sample <the 23 sampled files>
```

Classification rules, in order of precedence for a tactic line: the head token determines the family for `push_cast`-like heads (CAST), decision procedures (SIDE), normalisers (NORM), filter tactics (ANALYSIS); a `have`-like head is OBLIG if the line ends in `:= by <decision procedure>` and ASSERT otherwise; a `rw`/`simp`/`exact`/`refine`/`apply`/reshaping head is classified by the lemma names on the line, matched against cast, index, analysis, and natural-arithmetic patterns, and otherwise falls to REWRITE, STRUCT, or REPR by head. Continuation lines are not counted. Comments are stripped before parsing. Theorems are detected by a regular expression on `theorem`/`lemma`; a proof is term-mode if its first two lines contain no `by`.

Limitations are stated in Section 3. The single most consequential is that `rw` and `simp` lines without a recognised lemma pattern are counted as REWRITE regardless of what they do; the reconstructible share is therefore a lower bound.

Other artifacts in `docs/unified_report`: `feature_matrix.csv` (Table 8 in machine-readable form), `README.md`, and `build.sh`. The document builds with `latexmk -pdf` on a standard TeX distribution; the bibliography is embedded and no fonts beyond Latin Modern are required.

References

[1] *MathStep: A Proof Language of Mathematical Steps*. Design report, 5 September 2026. `docs/ideas/MathStep/MathStep.tex`.

[2] *Contour: A Human-Scale Proof Language over Lean*. Design report, 5 September 2026. `docs/ideas/contour-proof-language/contour.tex`.

[3] *Loom: A Contract-Based Language for Human-Scale Formal Proofs*. Design report, 5 September 2026. `docs/ideas/loom_proof_language/loom.tex`.

[4] *Mathematical Intent, Formal Evidence: A Proposal for a Mathematical Proof Layer over Lean*. Design report, 5 September 2026. `docs/ideas/mathematical_proof_language/article.tex`.

[5] *Mosaic: Mathematical Intent, Explicit Obligations, and Kernel-Checked Proof*. Design report, 5 September 2026. `docs/ideas/mosaic_proof_language/mosaic.tex`.

[6] *Motive: Concise Mathematical Reasoning with Kernel-Checked Elaboration*. Design report, 5 September 2026. `docs/ideas/motive_proof_language_article/motive.tex`.

[7] *Proofs at the Scale of Ideas: An Intent-Directed, Lean-Checked Proof Language*. Design report, 5 September 2026. `docs/ideas/outline_proof_language/outline_proof_language.tex`.

[8] *Reason: A Proof Language of Mathematical Intent*. Design report with executable supplement, 5 September 2026. `docs/ideas/reason_proof_language/reason.tex`.

[9] *Spine: A Proof Language for Mathematical Intent*. Design report, 5 September 2026. `docs/ideas/Spine_Proof_Language/Spine_Proof_Language.tex`.

[10] Vladimir Reshetnikov and contributors. *ProveIt*. Local checkout at commit `7c4e3f109405b9805b35d27ae96bf09c7ee5f3d5`; the reports pin `24ce8bd743eaab64a91ce90725ea00f498d319d2`. https://github.com/VladimirReshetnikov/ProveIt.

[11] Vladimir Reshetnikov and contributors. *Leant: a Djex-based synthesis REPL for Lean 4*. Local checkout at commit `3a40904be8a410d832d9ab6900b3d3e7b425eccb`, including `src/Leant/Synth/Verification.hs`, `src/Leant/Synth/Fragment.hs`, `docs/synth-internals.md`, `docs/candidate-quality.md`, and `docs/Leant_Djex_Lean4_Rewrite_Analysis`. https://github.com/VladimirReshetnikov/Leant.

[12] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *CADE 28*, LNCS 12699, pp. 625–635, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.

[13] The Lean development team. *Validating a Lean Proof*, in the Lean Language Reference. https://lean-lang.org/doc/reference/latest/ValidatingProofs/. Consulted by all nine reports on 5 September 2026.

[14] Mathlib contributors. *Mathlib.Tactic.Says*: the `says` combinator. Verified present in the Mathlib package pinned by ProveIt (toolchain `v4.32.0`). https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Says.html.

[15] Mathlib contributors. *Mathlib.Tactic.GCongr*, *Positivity*, *Bound*, *FunProp*, *Zify*, *Qify*, *WLOG*, and the unused-tactic and `haveLet` linters. Verified present in the same Mathlib package. https://leanprover-community.github.io/mathlib4_docs/.

[16] Jannis Limperg and Asta Halkjær From. Aesop: white-box best-first proof search for Lean. In *CPP 2023*, pp. 253–266. https://doi.org/10.1145/3573105.3575671.

[17] Patrick Massot. Teaching mathematics using Lean and controlled natural language. In *ITP 2024*, LIPIcs 309, 27:1–27:19. https://doi.org/10.4230/LIPIcs.ITP.2024.27.

[18] Adrian De Lon et al. The Isabelle/Naproche natural language proof assistant. In *CADE 28*, LNCS 12699, pp. 614–624, 2021. https://doi.org/10.1007/978-3-030-79876-5_36.

[19] Freek Wiedijk. A synthesis of the procedural and declarative styles of interactive theorem proving. *Logical Methods in Computer Science* 8(1:30), 2012. https://doi.org/10.2168/LMCS-8(1:30)2012.

[20] Makarius Wenzel. *Isar: Intelligible semi-automated reasoning*. https://isabelle.in.tum.de/Isar/.